

## Track 1 — 악성 컨트랙트 다크 패턴:

## 문헌과 분류체계

2026년 9월 19일

상태: 연구 노트 (detector spec의 입력. spec 자체는 아님)  
 날짜: 2026-09-19  
 범위: TRUST404 Track 1, *Smart Contract Threat Detection* — 오프라인 .sol 디렉터리 입력,  
 Benign / Malicious / Uncertain + 근거 JSON 출력  
 앵커: [trust404.co.kr/tracks](https://trust404.co.kr/tracks)

## 0. 트랙이 채점하는 것

트랙 페이지에 적힌 제약이다. 아래 내용 전체를 규정한다.

- 입력: .sol 소스. **디렉터리 일괄 처리가 필수**, 단일 파일은 선택.
- 출력: 분명한 판정 (Benign / Malicious / Uncertain) + **근거** (함수·코드 위치 + 이유) + 기계가 읽는 JSON.
- **오프라인 채점**. 네트워크 없음. 외부 API / LLM 호출은 실행되지 않는다. 모델과 규칙은 제출물 안에 넣는다.
- 비대화형 진입점 (CLI / 스크립트 / `docker run`).
- 채점 기준: 모의 컨트랙트 (공개 + 비공개 세트). **로직 흐름과 권한 구조에서 위험을 식별할 것, 키워드 매칭이 아님**. 타당한 추론: “비정상적 권한 이전, 은닉된 자산 인출 경로”.

이 두 한국어 구는 아래 리그풀 분류에서 그대로 쓰이는 범주 이름이다 (*Ownership Fraud*; *Leaking Token* / *Funds Manipulation*). 문제 부류는 **스캠 컨트랙트 탐지**이지, 취약점 탐지가 아니다.

## 1. 문헌이 합의하는 문제 설정

하위 문헌이 셋이다. 모두 **배포자 의도**를 다루며, 실수는 다루지 않는다.

| 하위 분야        | 핵심 질문                          | 시기    |
|--------------|--------------------------------|-------|
| 허니팟          | 사용자·공격자가 입금하게 유인한 뒤 자금을 가둔다    | 2019– |
| 토큰 백도어 / 리그풀 | 특권 함수로 배포자가 동결, 인출, 발행, 과세를 한다 | 2022– |
| 트랩도어         | “살 수는 있어도 팔 수는 없다”             | 2023– |

강한 도구는 모두 하나의 구조 질문으로 줄어든다.

사용자의 **전송 / 이탈 경로**에 놓인 **상태 변수**, 또는 **잔액을 직접 건드리는 상태 변수**에, **특권 기록 함수**가 있는가?

특권 + 이탈 경로 설계이다. 논문들이 이를 검증하고 정확한 술어를 준다 (§5).

## 2. 논문 지도

### 2.1 기초

**HoneyBadger** — “The Art of the Scam” · Torres, Steichen, State · USENIX Security 2019

[PDF](#) · [arXiv](#)

최초의 허니팟 분류. 기법 8종을 EVM / Solidity 컴파일러 / Etherscan 세 층으로 나눈다. 비트코드 심볼릭 실행. 허니팟 690건, 수동 검증 정밀도 87%. **부록 표 5에 이름이 붙은 허니팟 24개와 주소가 있다.** 픽스처로 쓸 수 있다.

**Pied-Piper** · Ma et al. · TOSEM 2022

[ACM](#) · [repo](#) · [tool](#)

ERC 토큰 백도어 다섯 가지: *Arbitrary Transfer, Generate Token After ICO, Destroy Token, Disable Transferring, Freeze Account*. Datalog + 방향 퍼징. 컨트랙트 13,484개 → 확정 189건, CVE 4건 (CVE-2019-16944 포함). **백도어를 심은 컨트랙트 200개**를 테스트셋으로 만들었다. 픽스처로 적합하다.

**TokenScope** · Chen et al. · CCS 2019

[PDF](#)

저장소 변화, 인터페이스 행동, 이벤트 사이의 불일치를 잡는다 (Transfer 없는 mint, 거짓을 말하는 balanceOf). 정밀도 >99.9%. “호출자에 따라 갈리는 view 함수” 검사의 근거가 된다.

### 2.2 리그풀 정적 탐지 (이번 빌드와 가장 가깝다)

**CRPWarner** · Lin et al. · IEEE TSE 2024

[arXiv](#) · [repo](#) + [datasets](#)

실제 리그풀 103건 → 컨트랙트 관련 유형: **Hidden Mint Function, Limiting Sell Order, Leaking Token** (무한 수수료 포함). 비트코드 Datalog 규칙. 오픈소스 스캠 컨트랙트 69건에서 P 91.8 / R 85.9 / F1 88.7. 대규모에서 정밀도 84.9% (13,484건 중 4,168건 표시). 거짓양성 분석이 명시되어 있다 (§6). 규칙 (Formula 1–4):

- Hidden Mint:  $\text{PublicFuncForOwner}(f) \wedge \text{LoadAndStoreBalances}(f) \wedge \neg \text{CheckBalances}(f)$
- Limiting Sell:  $\text{PublicFuncForOwner}(f) \wedge \text{VarToLimitTransfer}(v) \wedge \text{FuncModifyStorage}(f, v)$
- Leaking Token:  $\text{PublicFuncForOwner}(f) \wedge \text{FuncTransfer}(f) \wedge \text{CheckBalancesOfInput}(f)$
- Unlimited fee:  $\text{PublicFuncForOwner}(f) \wedge \text{VarForFee}(v) \wedge \text{FuncModifyStorage}(f, v)$  with no bound

**Tokeneer** — “Stop Pulling my Rug” · Zhou, Sun, Ma, Chen, Yan, Jiang (Tsinghua) · ICSE-SEIP 2024

[PDF](#) · [ACM](#) · [repo](#) [github.com/TokenSecure/Tokeneer](https://github.com/TokenSecure/Tokeneer)

사건 201건 (\$425M) → 위험 네 가지와 기저율: **BlackList 58.7%, ModifyBalance 42.3%**, TimeLimit, AlienDepend (외부 컨트랙트가 전송을 통제). 전송을 하위 과정 4개 (CallPublicTransfer → EnterInternalTransfer → SenderBalanceReduction → ReceiverBalanceAddition)와 플러그인 5개 (AddrCheck, TimingCheck, InternalCall, ExternalCall, BreakIn)로 모델링하고, 오라클을 도달 가능한 경로의 어디에든 맞춘다. 재현율 98.0% / 정밀도 98.9%. GoPlus·Pied-Piper보다 실제 위험을 27.2% 더 찾았다. 핵심: Pied-Piper와 GoPlus는 `_transfer`에 들어가기 전에 놓인 검사 (Dialectic)나 헬퍼에 감싼 검사 (getCheckRoot in TMK)를 놓친다.

**TrapdoorAnalyser** — “From Programming Bugs to Multimillion-Dollar Scams” · Huynh et al. · 2023, rev. Dec 2024

[arXiv v4](#)

트랩도어 토큰 1,859개를 수동 검사. 기법 다섯 가지: **Exchange Permission, Exchange Suspension, Amount Limit, Fee Manipulation, Invalid Callback**. 의미 검사는 Slither AST를 Python API로 돌린다. 이번 프로젝트와 같은 스택이다. 라벨된 UniswapV2 토큰 약 3만 개 (트랩도어 11,943 / 비(非)트랩도어 18,548). GoPlus는 이 트랩도어의 60%만 잡았다.

의미 검사가 모으는 것: 상태 변수  $s$  (address / int / bool / const), 전송 함수  $F$ 와 그것이 부르는 모든 것, 전송 입력  $I = \{sender, receiver, amount\}$ ,  $F$ 에서 쓰이는 끝 노드  $N$  (require / assert / revert / return), 백도어 함수  $B$  (호출자를 저장된 주소 또는 목록과 비교), 외부 호출  $c$ . 지표는  $s$ 의 변수가  $F$ 의 끝 노드 (또는 amount 계산)에서 쓰이면서 동시에  $B$  또는  $c$ 에서 쓰일 수 있는 것이다.

**RPHunter — “Your Token Becomes Worthless”** · Wu et al. · 2025

[arXiv](#) · [dataset \(IEEE DataPort\)](#) · [Figshare](#)

가장 큰 수동 코퍼스: 소스가 있는 리그풀 645건. 코드 위험 분류, 3범주 / 8하위유형, 건수:

- Sale Restrict (203): Amount Restrict, Timestamp Restrict, Address Restrict
- Variable Manipulation (144): Modifiable Tax Rate, Modifiable Tax Address, Modifiable External Call
- Balance Tamper (189): Hidden Mint/Burn, Hidden Balance Modification

**Detecting Rug-Pull via balance tracking** · Applied Sciences 2025

[MDPI](#)

바이트코드에서 함수별 잔액 변화를 추적. 백도어 여섯 유형: 토큰 생성, 소각, 전송 제한, 자금 조작, 전송 수수료, 프록시. 정밀도 0.98 / 재현율 0.96.

**Geth-based ERC20 honeypot detector** · Discover Computing 2025

[Springer](#)

테인트: sender/to로 색인된 boolean 저장소가 revert 분기를 결정하면 허니팟. Pied-Piper 백도어 189건 중 transfer를 건드리지 않는 65건 (mint / destroy / arbitrary transfer)은 놓쳤다. 이탈 차단만으로는 부족하다는 경고이다.

## 2.3 분류체계와 도구 감사

**SoK: A Taxonomic Analysis of DeFi Rug Pulls** · Sun, Ma, Nie, Liu (NTU) · ISSTA 2025

[PDF](#) · [ISSTA page](#)

논문 31편 + 업계 자료 27건 → 리그풀 유형 35개, 그중 새로운 9개 (Hidden Mint, Burn, Balance Modification, Ownership Transfer, Hidden Owner, Unverified Contract, External Call, Liquidity Pool Block, Fake LP Lock). 도구 13개를 평가: 커버리지 25.7%–62.9%. 어떤 도구도 못 잡은 유형 9개. 복합 스캠에서 최고 도구는 31.3%로 떨어진다. 사례 DOGE3.0: 프록시로 소유권을 숨기고, \_sender 숨은 거버넌스, “burn”이 실제로는 mint. 정적 도구의 실패 모드 넷을 이름 붙인다 (§4).

**SoK: Rug Pull Causes, Datasets, Tools** · 2024 (earlier version)

[arXiv](#)

근본 원인 34개. 어떤 도구도 커버하지 못한 유형: Fake LP Lock, Hidden Fee, Destroy Token, Fake Money Transfer, Ownership Transfer, Liquidity Pool Block, Freeze Account, Wash-Trading, Hedge.

**ObfProbe — “Obfuscated Funds Transfers”** · 2025

[arXiv](#)

컨트랙트 103만 개의 전송 로직에서 바이트코드 난독화 특징 일곱 가지. 강하게 난독화된 것 3,128건. SOTA 폰지 탐지기가 난독화 아래 79% → 12%로 무너졌다. 가짜 renounceOwnership(), failsafe() / emergency() 백도어, isTokenReceiver 숨은 소유자를 문서화한다.

**Static analysis of NFT rug pulls** · 2025 · [arXiv](#)

NFT 컨트랙트 4.99만 개에 Slither를 대규모로 돌림. 가장 많이 표시된 구문: owner-only withdraw, 제한 없는 mint, selfdestruct, delegatecall, tx.origin 인증, 기만적 이름 (safeWithdraw).

## 2.4 트랜잭션 측 (맥락만. 오프라인 채점에는 쓸 수 없음)

- **Trade or Trick?** · Xia et al. · SIGMETRICS 2022 · [arXiv](#) — 스캠 토큰 10,920개 ≈ Uniswap V2의 50%. 연좌 + 트랜잭션 ML.
- **Token Spammers, Rug Pulls, and Sniper Bots** · Cernera et al. · USENIX Security 2023 · [page](#) · [dataset](#) — 토큰의 60%가 수명 1일 미만. “1-day rug pull” ≈ \$240M.
- **Do Not Rug On Me** · Mazorra, Adan, Daza · 2022 · [arXiv](#) — 풀 / 홀더 / 트랜잭션 그래프 특징 ML, 토큰 27,588개.
- **Detecting Ponzi Schemes on Ethereum** · Chen et al. · WWW 2018 · [PDF](#) — 옴코드 빈도 + 계정 특징, XGBoost.

## 2.5 업계 체크리스트 (경쟁 팀이 그대로 베낄 것)

- **GoPlus token security fields** · [docs](#): is\_honeypot, hidden\_owner, can\_take\_back\_ownership, is\_mintable, slippage\_modifiable, personal\_slippage\_modifiable, transfer\_pausable, is\_blacklisted, is\_whitelisted, is\_anti\_whale, trading\_cooldown, is\_proxy, external\_call, selfdestruct, owner\_change\_balance.
- **TokenSniffer exploit typologies** · [docs](#): honeypot, blocklist/allowlist, hidden mint, balance modification, fake ownership renounce, fee modifier. Tests: testForProxy, testForPausable, testForMint, testForRestoreOwnership, testForMaxTransactionAmount, testForModifiableFee, testForBlacklist, testForOwnershipNotRenounced.
- **ethereum.org token integration checklist** · [source](#) — “Owner privileges”: 업그레이드 불가, mint 제한, pause 불가, 블랙리스트 불가.
- **Ice phishing / drainers** · [Forta](#) · [Check Point](#) — 가짜 클레임 페이지로 approve / permit / setApprovalForAll을 수집한 뒤, 같은 블록에서 transferFrom. 종종 multicall.

## 3. 통합 다크 패턴 분류

HoneyBadger, Pied-Piper, CRPWarner, Tokeer, TrapdoorAnalyser, RPHunter, ISSTA SoK, GoPlus, TokenSniffer를 합쳤다. 공격자가 얻는 것으로 묶었다. 괄호 안의 실세계 토큰 예는 인용 논문에서 왔다.

### A. 이탈 차단 — Sale Restrict / Limiting Sell Order / Trapdoor

가장 흔한 계열. Tokeer 사건의 58.7%. RPHunter 645건 중 203건.

| 패턴          | 형태                                                                                                                     | 예시                                          |
|-------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 주소 제한       | 전송 경로의 require/if/revert에서 mapping (address=>bool)을 읽음. 특권이 씬. 이름은 거짓말: _isBot, isExcluded, _antisswaplist, _safeOwner | AquaDrops, Sirius_Finance                   |
| 전역 중단       | bool (tradingEnabled, transfersEnabled, paused, 또는 한 글자 이름)이 전송을 막음. 특권이 씬. 공개 해제 함수가 없으면 더 나쁨                         | Connective, Bancor (정상 사용)                  |
| 수량 한도       | 끝 노드에서 maxTxAmount / maxWallet을 amount와 비교. 특권이 쓰며 하한이 없음                                                              | EVGR (1로 설정)                                |
| 시간 제한 / 쿨다운 | block.timestamp를 특권이 쓰는 한도와 비교                                                                                         | Tokeer TimeLimit                            |
| 매도만 차단      | to == uniswapV2Pair (또는 특권이 쓰는 주소 변수)일 때만 막힘. 매수는 되고 매도는 revert                                                        | 대부분의 트랩도어                                   |
| 잘못된 콜백      | 전송이 중첩 호출을 일으켜, 통제된 주소로 게이트에 재진입. 매도자는 목록에 없어도 매도가 항상 revert                                                           | ELONAJA (burnToken → _transfer(token, ...)) |

| 패턴      | 형태                                                       | 예시                                         |
|---------|----------------------------------------------------------|--------------------------------------------|
| 수수료 무기화 | 수수료 변수로 amount를 계산. 특권이 씌. 상한 없음. 극단값이 언더플로를 일으켜 버그처럼 보임 | 88 Dollar Millionaire (99%), CPP4U (1000%) |

## B. 잔액 조작 — *Hidden Mint / Balance Modification / ModifyBalance*

Tokeer 사건의 42.3%. RPHunter 645건 중 189건.

| 패턴           | 형태                                                                                 | 예시                                  |
|--------------|------------------------------------------------------------------------------------|-------------------------------------|
| 특권 발행, 임의 대상 | 특권 함수에서 <code>balances[x] += n / totalSupply += n</code> . 충분성 검사 없음, 상한 없음, 생성자 박 | Pokemoney / NEKOGOLD, TEDDY         |
| 소각으로 위장한 발행  | 조건이 맞으면 소유자 잔액을 늘리는 “burn”                                                         | DOGE3.0                             |
| 전송 안의 발행     | 전송 경로에서 입금이 출금보다 큼. “특수” 수신자에 조건                                                   | ISSTA SoK Listing 2                 |
| 호출자 의존 view  | <code>balanceOf / totalSupply</code> 가 <code>msg.sender</code> 로 갈림 (이중 상태)        | ISSTA SoK Listing 3; TokenScope 부류  |
| 타인 잔액 특권 소각  | 특권 함수에서 <code>balances[from] -= n</code> . <code>from</code> 이 임의                  | Pied-Piper type 3, CVE-2019-16944   |
| 잔액 직접 대입     | 특권 함수에서 <code>balances[x] = v</code>                                               | TokenSniffer “balance modification” |

## C. 자금 추출 — *Leaking Token / Arbitrary Transfer / Funds Manipulation*

트랙의 “은닉된 자산 인출 경로”.

| 패턴              | 형태                                                                                                                                                                                                          | 예시                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| 임의 transferFrom | 특권 함수가 allowance 없이 아무 계정의 토큰을 from에서 옮김                                                                                                                                                                    | Pied-Piper zero_fee_transaction; CRPWarner Leaking Token |
| 특권 면제 경로        | <code>_transfer</code> 에서 특권 송신자에 대해 잔액 검사 / 차감을 건너뛴                                                                                                                                                        | ZHONGHUA                                                 |
| 특권 스왑           | 컨트랙트가 들고 있는 ETH/토큰을 밖으로 보냄. 이름: <code>emergencyWithdraw, failsafe, rescue, safeWithdraw</code>                                                                                                              | Gold Mine Finance                                        |
| 바꿀 수 있는 수수료 수신처 | 수수료·세금 주소를 특권이 씌                                                                                                                                                                                            | RPHunter MTA                                             |
| 허니팟 미끼 (레저시)    | 입금은 받고, 이탈 경로에 숨은 owner-only 조건: Hidden Transfer, Straw Man Contract, Balance Disorder, Inheritance Disorder, Skip Empty String Literal, Type Deduction Overflow, Uninitialised Struct, Hidden State Update | HoneyBadger Table 5                                      |

## D. 제어면 기만 — *Ownership Fraud*

트랙의 “비정상적 권한 이전”.

| 패턴           | 형태                                                                                                                                                 | 예시                                                         |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 숨은 소유자       | 두 번째 주소 상태 변수 ( <code>_sender, isTokenReceiver, _safeOwner, dev, marketingWallet</code> )를 인증에서 <code>msg.sender</code> 와 비교. owner는 겉으로 renounce됨 | DOGE3.0. <code>_sender</code> 를 잡은 도구는 5개 중 1개 (ISSTA SoK) |
| 가짜 renounce  | <code>renounceOwnership()</code> 이 이벤트만 내고 아무것도 안 하거나, 다른 공격자 주소로 옮기거나, 숨은 역할을 남김                                                                  | ObfProbe 사례                                                |
| 소유권 회수       | 비표준 함수가 owner를 대입. 배포 후 initialize                                                                                                                 | GoPlus can_take_back_ownership                             |
| tx.origin 인증 | <code>tx.origin</code> 으로 인가                                                                                                                       | NFT 리그풀 연구                                                 |
| 불완전한 포기      | renounce 전에 두 번째 특권 주소를 추가                                                                                                                         | DYDZ                                                       |

### E. 구조적 탈출구 — *AlienDepend / Modifiable External Call / Proxy*

| 패턴                               | 형태                                                                                                                         | 주                                                                                        |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 외부 게이트                           | 전송 게이트·로직을 외부 컨트랙트에 위임. 그 주소는 특권이 씀<br>( <code>_antiBot.getAntiBotEnable()</code> , <code>botProtection</code> 의 헤스 이름 함수) | 오프라인에서 해석 불가 → <b>이유를 불인</b><br><b>Uncertain</b> . Benign이 되면 안 됨 (Elongate Deluxe, YZZ) |
| 설정 가능한 <code>delegatecall</code> | 대상을 특권 역할이 씀                                                                                                               |                                                                                          |
| 업그레이드 프록시, EOA 관리자               | 단일 키 업그레이드 권한                                                                                                              | DOGE3.0은 프록시로 소유권을 숨김                                                                    |
| <code>selfdestruct</code>        | 있고 도달 가능                                                                                                                   |                                                                                          |

### F. 승인 피싱 / 드레인 컨트랙트

트랙 사용자가 서명 팝업 앞에 있기 때문에 관련이 있다.

| 패턴           | 형태                                                                                                                                                                                                                            |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 클레임/에어드롭 드레인 | 실제 효과는 <code>msg.sender</code> 로부터 <code>transferFrom/permitTransferFrom/safeTransferFrom</code> 을 하드코드 또는 소유자가 정한 주소로 보냄. 종종 <code>multicall</code> . 호출자에게 대응하는 입금 없음. 셀렉터: <code>0x23b872dd, 0xd505accf, 0xa22cb465</code> |

### G. 폰지 / 사기 구조 컨트랙트

앞선 입금자에게 주는 지급이 이후 `msg.value`로만 채워진다. 외부 가치 원천이 없다. 모의 세트에 있으면 값싼 구조 검사. 우선순위는 낮다.

## 4. 함정이 숨는 곳 — 키워드·셀렉터 매칭이 실패하는 이유

TrapdoorAnalyser §3.6, Tokeer §3, ObfProbe, ISSTA SoK.

#### 위치

- 시그니처 중간에 붙고 다른 컨트랙트에 정의된 `modifier` (`YearnLending.Finance onlyPermitted`).
- 헬퍼 2-3층 아래 (`getCheckRoot`). 중첩 함수에 걸쳐 계산되는 수수료 (`The Reckoning Force`).
- `_transfer`에 들어가기 전에 놓인 검사 (`Dialectic`).
- 외부 컨트랙트 (`Elongate Deluxe`).

#### 이름과 결모습

- 블랙리스트 이름 `_isBot`. 페어 주소를 “zero address”로 명명. 한 글자 스위치 (`i, t, l`).
- 백도어에 무해한 이름: `failsafe, emergency, isTokenReceiver, safeWithdraw`.
- 변수를 올바르게 설정할 `init`이 있으나 결코 호출되지 않음 (`AIRSHIB`).
- 빈 `revert` 메시지.
- `require` 대신 수치 예외를 써서, 실패가 정직한 버그처럼 읽힘.

#### 코드 구조 (ObfProbe의 특징 일곱 가지)

- 죽은 코드, 깊은 함수 분할, 인라인 어셈블리 / `raw sstore`, 주소 계산용 문자열 연산, 전송 함수 안의 낮은 전송 명령 비율.

#### 런타임 조건 활성화

- `block.timestamp`, 특수 수신자, 또는 프록시의 현재 `implementation`으로 막힘.

#### ISSTA SoK가 말한 정적 도구의 실패 모드 넷

1. 동적 맥락 맹점 (시각에 묶인 행동)
2. 숨은 변수 해석 (동적으로 대입된 소유자 주소)
3. 복잡한 조건 논리 (다중 조건 게이트)

#### 4. 거버넌스와 기능의 분리 (검사와 잔액 연산이 다른 함수에 있음)

수렴하는 수정 (Tokeer, TrapdoorAnalyser, RPHunter): `transfer / transferFrom`에서 도달 가능한 호출 그래프 (modifier와 사전 검사 포함)로 전송 경로를 모델링한 뒤, 무엇을 데이터가 막고 누가 그 데이터를 쓰는지 묻는다. 이름을 맞추지 않는다.

### 5. 통합 공식 → 구현 술어

강한 도구는 모두 같은 삼중으로 줄어든다.

- 특권 기록 함수  $f$  — 도달이 `msg.sender == <상태 주소 변수>, <address→bool 맵>[msg.sender]`, 또는 `tx.origin`에 달려 있는 `public/external` 함수. modifier나 인라인 검사 어디든. 이름에 의존하지 않음 (숨은 소유자를 잡음).
- 막히거나 변조된 상태  $v$  —  $f$ 에서 쓰인 상태 변수.
- 영향 —  $v$ 가
  - (a) 전송 경로의 `require / if / revert` 끝 노드에서 읽힘 → 이탈 차단
  - (b) 전송액 또는 수수료 계산에 쓰임 → 수수료 조작
  - (c) 잔액 / `totalSupply` 매핑 그 자체이고 직접 쓰임 → 잔액 조작
  - (d) 전송 경로의 외부 호출 / `delegatecall` 대상 주소 → 외부 의존 (alien dependence)

Slither (Python API)가 술어를 모두 공급한다.

| 술어                             | Slither 표현                                                                                                                                             |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 특권 기록 함수                       | <code>Function.is_protected()</code> , modifier 본문, <code>require/if</code> 노드의 <code>msg.sender / tx.origin</code> 비교, <code>msg.sender</code> 데이터 의존 |
| 쓰인 상태                          | <code>Function.state_variables_written</code> , <code>all_state_variables_written()</code>                                                             |
| 전송 경로                          | <code>transfer / transferFrom</code> 과 그 modifier에서 도달 가능한 호출 그래프 ( <code>Function.all_internal_calls()</code> , <code>Function.modifiers</code> )     |
| 끝 노드                           | <code>require / assert / revert (SolidityCall)</code> 를 담은 노드, <code>if</code> 조건                                                                      |
| 금액 의존                          | <code>is_dependent(var, amount_param, function)</code>                                                                                                 |
| 잔액 매핑                          | <code>balanceOf</code> 가 읽고 전송에서 차감하는 그 매핑 (뭉어서 볼 것. 아무 <code>address→uint</code> 나 잔액으로 보지 말 것)                                                       |
| 외부 / <code>delegatecall</code> | SlithIR <code>HighLevelCall</code> , <code>LowLevelCall</code> , <code>delegatecall</code>                                                             |
| 거짓말하는 view                     | 반환이 <code>msg.sender</code> 에 의존하는 view 함수                                                                                                             |

TrapdoorAnalyser가 Slither 위에 바로 이것을 만들었다. 이번 프로젝트는 공개된 설계를 다시 구현하되, 판정 층을 더 분명하게 하고 오프라인으로 패키징한다.

### 6. 정상처럼 보이는 것들 — Uncertain으로 두고 Malicious로 올리지 않을 것

GoPlus를 베낀 경쟁 팀이 정상 샘플에서 점수를 잃는 지점.

| 유사 형태                                   | 혼동 이유                                                                                   | 판별 기준                                                          |
|-----------------------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------|
| <code>_burn</code> vs <code>mint</code> | CRPWarner 거짓양성: 잔액 검사 없음 + 컴파일러가 SUB를 ADD로 재작성                                          | 소스 수준 연산자가 보인다. 그것을 쓸 것. <code>mint</code> 는 대상을 늘린다           |
| <code>excludeFromFee</code> vs 불랙리스트    | 같은 형태: <code>address→bool</code> , <code>owner</code> 가 씀, <code>_transfer</code> 에서 읽음 | 그 읽기가 어디로 가는지: 수수료 금액 (정상) vs <code>revert</code> 하는 끝 노드 (차단) |
| DEX 페어 검사 vs <code>onlyOwner</code>     | <code>to == uniswapV2Pair</code> 가 주소 비교처럼 보임                                           | <code>msg.sender / tx.origin</code> 비교만 인증이다                   |

| 유사 형태                                                      | 혼동 이유                                                                                          | 판별 기준                                                                                        |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| address→uint 상태 맵                                          | 잔액으로 오인                                                                                        | “잔액 매핑”을 balanceOf가 읽는 그 하나에 묶을 것                                                            |
| OpenZeppelin Pausable, Ownable, ERC20Capped, AccessControl | 정상이고 혼하다. Lido LDO는 transfersEnabled+owner-only enableTransfers. Bancor는 disableTransfers를 실었다 | 공개 해제와 함께 있는, 범위가 정해진 단일 특권 → Uncertain / 주식 있는 Benign. 무한, 숨은 소유자, 또는 잔액 조작과 결합 → Malicious |
| 안티웨일 / 안티봇 런치 한도                                           | Amount Limit와 같은 변수                                                                            | 하드코드 하한 + 시한 창 → 정상 형태. 하한 없음, 만료 없음 → 트랩도어                                                  |
| 외부 토큰 회수                                                   | 스왑처럼 보임                                                                                        | 컨트랙트가 스스로 발행하거나 사용자를 위해 들고 있는 토큰의 스왑이 악성 경우                                                  |

기저율로 맞추면: 모든 코퍼스에서 사고의 대부분은 이탈 차단과 잔액 조작이다. 수수료만, pause만 있는 탐지 항목이 정상 샘플에 가장 자주 있다.

## 7. 초안 규칙 집합 (detector spec용)

심각도: **HIGH** → Malicious를 이끈다. **MED** → Uncertain을 이끈다. **INFO** → 근거만.

| 규칙 ID                    | 계열           | 트리거                                                                        | 심각도                                              |
|--------------------------|--------------|----------------------------------------------------------------------------|--------------------------------------------------|
| PRIV_ROLE                | meta         | 이름에 의존하지 않는 특권 역할: 인증에서 msg.sender/tx.origin을 저장된 주소 또는 address→bool 맵과 비교 | — (다른 규칙에 공급)                                    |
| EXIT_ADDR_GATE           | A            | 특권이 쓰는 address→bool을 전송 경로 끝 노드에서 읽음                                       | HIGH                                             |
| EXIT_GLOBAL_SWITCH       | A            | 특권이 쓰는 bool이 전송 경로를 막음                                                     | 공개 해제가 없으면 HIGH. unpause가 있는 OZ-Pausable 형태면 MED |
| EXIT_AMOUNT_LIMIT        | A            | 특권이 쓰는 숫자를 끝 노드에서 amount와 비교                                               | 하한이 없으면 HIGH. 상수 하한이면 MED                        |
| EXIT_TIME_GATE           | A            | 전송 경로에서 block.timestamp를 특권이 쓰는 한도와 비교                                     | MED. 만료가 없으면 HIGH                                |
| EXIT_SELL_ONLY           | A (modifier) | to가 특권이 쓰는 주소 변수와 같을 때만 막힘                                                 | 한 단계 상향                                          |
| EXIT_CALLBACK_CYCLE      | A            | 전송 경로의 호출 순환. 입력을 끝 노드에서 특권이 쓰는 변수와 비교                                     | HIGH                                             |
| FEE_UNBOUNDED            | A/C          | 특권이 쓰는 수수료 변수가 amount에 들어가고, 상한 ≤ 25%가 없음                                  | HIGH. 상한이 있으면 MED                                |
| FEE_ADDR_MUTABLE         | C            | 특권이 쓰는 수수료 수신처                                                             | MED                                              |
| BAL_PRIV_MINT            | B            | 특권이 잔액/totalSupply를 늘림. 임의 대상, 상한 없음, 생성자 밖                                | HIGH. ERC20Capped 형태면 MED                        |
| BAL_PRIV_BURN_OTHER      | B            | 특권이 다른 계정의 잔액을 줄임                                                          | HIGH                                             |
| BAL_DIRECT_SET           | B            | 특권이 잔액 매핑에 직접 대입                                                           | HIGH                                             |
| BAL_TRANSFER_HIDDEN_MINT | B            | 전송 경로에서 입금 > 출금                                                            | HIGH                                             |
| VIEW_CALLER_DEPENDENT    | B            | balanceOf / totalSupply 반환이 msg.sender에 의존                                 | HIGH                                             |

| 규칙 ID                        | 계열  | 트리거                                                                                | 심각도                                           |
|------------------------------|-----|------------------------------------------------------------------------------------|-----------------------------------------------|
| LEAK_ARBITRARY_TRANSFERFROM  | C   | 특권이 allowance 없이 임의 계정에서 이동                                                        | HIGH                                          |
| LEAK_EXEMPT_PATH             | C   | 전송의 특권 분기가 차감/점사를 건너뛴                                                              | HIGH                                          |
| LEAK_PRIV_SWEEP              | C   | 특권이 컨트랙트가 든 ETH / 자기 토큰을 보냄                                                        | 사용자 자금을 보<br>관하면 HIGH. 외<br>부 토큰 회수만이면<br>MED |
| OWN_HIDDEN_ROLE              | D   | 보이는 owner가 아닌 인증 주소 변수                                                             | HIGH                                          |
| OWN_FAKE_RENOUNCE            | D   | renounceOwnership이 인증 변수를 모두 지우<br>지 않거나 재대입                                       | HIGH                                          |
| OWN_REASSIGN_NONSTD          | D   | 생성자 / 표준 transferOwnership 밖에서<br>owner 대입                                         | HIGH                                          |
| OWN_TX_ORIGIN                | D   | 인증에 tx.origin                                                                      | MED                                           |
| STRUCT_EXTERNAL_GATE         | E   | 전송 경로가 특권이 쓰는 주소로의 외부 호출에 의존                                                       | → 이유와 함께<br><b>Uncertain</b>                  |
| STRUCT_DELEGATECALL_SETTABLE | E   | 특권이 쓰는 대상으로 delegatecall                                                           | HIGH                                          |
| STRUCT_SELFDESTRUCT          | E   | selfdestruct에 도달 가능                                                                | HIGH                                          |
| STRUCT_PROXY_EOA_ADMIN       | E   | 단일 주소 관리자의 업그레이드 가능                                                                | MED                                           |
| DRAIN_APPROVAL_PULL          | F   | msg.sender로부터 transferFrom/<br>permit / safeTransferFrom을 호출자가 아닌<br>곳으로, 반대 입금 없이 | HIGH                                          |
| HONEYPOT_LEGACY_*            | C/F | HoneyBadger 패턴 (이탈의 숨은 owner 조건, straw<br>man 등)                                   | MED                                           |
| PONZI_SHAPE                  | G   | 저장된 입금자에게 주는 지급이 msg.value로만 채<br>워짐                                               | MED                                           |
| SLITHER_HIGH_OVERLAY         | —   | Slither 내장 High 탐지기                                                                | INFO. Benign을<br>Uncertain으로만 올<br>릴 수 있음     |

초안 판정 정책 (Discord 공개 샘플로 조정):

- HIGH가 하나라도 있으면 → **Malicious**
- MED만 있으면 → **Uncertain**
- STRUCT\_EXTERNAL\_GATE 또는 컴파일 실패 → reason과 함께 **Uncertain**
- 아무것도 없으면 → **Benign**
- 선택: 서로 다른 계열의 MED 두 개 (A + B, A + D) → **Malicious**

모든 탐지 항목은 contract, function, source\_lines, rule\_id, severity, reasoning을 단다.

## 8. 픽스처·튜닝용 데이터셋

| 출처               | 내용                              | 링크                                              |
|------------------|---------------------------------|-------------------------------------------------|
| Pied-Piper       | 백도어를 심은 컨트랙트 200개 + 실제 사례       | <a href="#">repo, tool</a>                      |
| CRPWarner        | 오픈소스 러그폴 69건 (정답) + 1.3만 대규모 세트 | <a href="#">repo</a>                            |
| Tokeneer         | 코드 + 데이터                        | <a href="#">github.com/TokenSecure/Tokeneer</a> |
| TrapdoorAnalyser | 라벨된 UniswapV2 토큰 약 3만 개         | 논문 §4.3의 링크                                     |
| RPHunter         | 러그폴 소스 645건                     | <a href="#">IEEE DataPort, Figshare</a>         |

| 출처             | 내용                           | 링크                           |
|----------------|------------------------------|------------------------------|
| ISSTA 2025 SoK | 29/35 유형에 걸친 2,391건          | 논문 아티팩트                      |
| HoneyBadger    | 이름이 붙은 허니팟 24개와 주소 (Table 5) | <a href="#">PDF</a>          |
| H6-Guard       | Solidity → JSON 판정 허니팟 데이터셋  | <a href="#">Hugging Face</a> |
| Cernera et al. | Ethereum/BSC 토큰 + LP 데이터셋    | <a href="#">repo</a>         |

우선순위: **Discord 공개 샘플이 먼저다.** 심사위원 모의의 스타일이다. 네트워크가 아직 있을 때 외부 픽스처를 받아 둔다. 채점은 오프라인이다.

## 9. 빌드에 대한 합의

방향은 그대로다. 논문이 정당화하는 추가 셋:

1. **이름에 의존하지 않는 특권 탐지** — 도달 가능한 인증에서 `msg.sender / tx.origin`과 비교되는 아무 주소 상태 변수 또는 `address→bool` 맵. SoK가 시험한 모든 도구의 가장 큰 공백.
2. **Token식 전송 경로 모델링** — `transfer / transferFrom`에서 도달 가능한 모든 함수·modifier를 모은다. `_transfer` 앞 검사 포함. 차단 조건은 그 집합에서 평가하고, `_transfer`만 보지 않는다.
3. **이유를 붙인 Uncertain** — 전송 경로의 해석되지 않은 외부 호출과 컴파일 실패에 명시한다. “로직을 볼 수 없다”가 점수가 되는 답이지, 조용한 Benign이 아니다.

소스만 받는 입력이 여기서 이점이다. CRPWarner, Token, RPHunter는 바이트코드를 디컴파일해야 했다. 이번 프로젝트는 TrapdoorAnalyser처럼 Slither AST/IR을 바로 받는다.

## 10. 결정 로그

- **제출 경로는 오프라인만.** Tenderly 없음, 메인넷 `eth_call` 없음, 포크 없음, 호스티드 모델 없음 (TypeSafe Jev 포함). Demo Day 이야기에 는 나와도 된다.
- **Slither는 라이브러리지, 제품이 아니다.** 판정 로직과 근거는 우리 것. Slither는 파서/IR.
- **시뮬레이션을 쓰면, 컴파일된 모의 컨트랙트 위의 로컬 Anvil/Foundry (특권 setX 다음 사용자 transfer). docker run --network none 안.**
- **Slither 내장 탐지기는 오버레이이지, Malicious의 구동기가 아니다.**